

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA
DEPARTAMENTO DE INFORMÁTICA
CIUDAD - CHILE



“ASISTENTE DE ACCESIBILIDAD EN SITIOS WEB PARA PERSONAS CON DISCAPACIDAD VISUAL MEDIANTE EL USO DE INTELIGENCIA ARTIFICIAL”

IGNACIO BENJAMÍN RIQUELME VIDAL

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN INFORMÁTICA

Profesor Guía: Pedro Felipe Toledo Correa
Profesor Correferente: Jose Luis Martí Lara

Julio - 2026

DEDICATORIA

Considerando la importancia de este trabajo para los alumnos, este apartado es para que el autor entregue palabras personales para dedicar este documento. La extensión puede ser de máximo una hoja y se deben mantener este formato, tipo y tamaño de letra.

AGRADECIMIENTOS

Considerando la importancia de este trabajo para los alumnos, este apartado se podrá incluir en el caso de que el autor desee agradecer a las personas que facilitaron alguna ayuda relevante en su trabajo para la realización de este documento. La extensión puede ser de máximo una hoja y se deben mantener este formato, tipo y tamaño de letra.

RESUMEN

Resumen— El resumen y las palabras clave no deben superar la mitad de la página, donde debe precisarse brevemente: 1) lo que el autor ha hecho, 2) cómo lo hizo (sólo si es importante detallarlo), 3) los resultados principales, 4) la relevancia de los resultados. El resumen es una representación abreviada, pero comprensiva de la memoria y debe informar sobre el objetivo, la metodología y los resultados del trabajo realizado.

- Relator de pantalla: prototipo de herramienta que interpreta el contenido visual de sitios web y responde por voz a personas con discapacidad visual.
- Metodología de trabajo: encuesta -> prototipo -> *benchmark* -> validación con usuarios.
- Relevancia: >2.200 millones de personas con deficiencia visual; los lectores de pantalla no interpretan lo visual.

Palabras Clave— Cinco es el máximo de palabras clave para describir los temas tratados en la memoria, ponerlas separadas por punto y comas.

accesibilidad web; discapacidad visual; VLM; lectores de pantalla; *benchmarking*.

ABSTRACT

Abstract— Corresponde a la traducción al idioma inglés del Resumen anterior. Sujeto a la misma regla de extensión del Resumen.

Keywords— Corresponde a la traducción al idioma inglés de Palabras Clave anteriores.

GLOSARIO

Aquí se deben colocar las siglas mencionadas en el trabajo y su explicación, por orden alfabético. Por ejemplo:

DI: Departamento de Informática.

UTFSM: Universidad Técnica Federico Santa María.

ÍNDICE DE CONTENIDOS

RESUMEN	IV
ABSTRACT	IV
GLOSARIO	V
ÍNDICE DE FIGURAS	VIII
ÍNDICE DE TABLAS	VIII
INTRODUCCIÓN	1
CAPÍTULO 1: DEFINICIÓN DEL PROBLEMA	2
1.1 PROBLEMÁTICA SOCIAL	2
1.1.1 Antecedentes y encuesta propia	3
1.2 PROBLEMÁTICA TÉCNICA	4
1.2.1 Funcionamiento de los lectores de pantalla	4
1.2.2 Dependencia de la semántica: ARIA y contenido dinámico	4
1.2.3 Límites del enfoque	5
1.2.4 Soluciones existentes y su insuficiencia	5
1.3 OBJETIVOS Y ALCANCES	5
CAPÍTULO 2: MARCO CONCEPTUAL	7
2.1 LECTORES DE PANTALLA Y EL DOM	7
2.2 EXTENSIONES DE NAVEGADOR	9
2.3 RECONOCIMIENTO Y SÍNTESIS DE VOZ (STT Y TTS)	9
2.4 IA MULTIMODAL: MODELOS DE LENGUAJE VISUAL (VLM)	10
2.4.1 IA generativa de lenguaje	10
2.4.2 IA multimodal	10
2.4.3 Entrenamiento	11
2.4.4 Inferencia	11
2.5 EVALUACIÓN AUTOMATIZADA DE VLMs	12
2.5.1 Rúbrica cualitativa	12
2.5.2 <i>LLM-as-a-judge</i>	12
2.5.3 <i>LLM-as-a-judge-evaluation</i>	13
2.5.4 Herramientas estandarizadas de evaluación	13
CAPÍTULO 3: PROPUESTA DE SOLUCIÓN	14
3.1 ARQUITECTURA GENERAL DEL RELATOR DE PANTALLA	14
3.2 EXTENSIÓN DE NAVEGADOR	16
3.2.1 <i>Scroll & stitch</i>	16
3.2.2 Formato de salida de la imagen	16
3.3 INTERACCIÓN CON EL USUARIO	17

3.3.1	Interacción sin síntesis de voz	17
3.3.2	Interacción con síntesis de voz	17
3.4	BACKEND DEL SISTEMA	17
3.5	INFRAESTRUCTURA Y SERVIDOR ALBERT	18
3.5.1	Decisión de infraestructura	18
3.6	BENCHMARK DE MODELOS VLM	20
3.6.1	Sitios y preguntas	20
3.6.2	Rúbrica de evaluación	20
3.6.3	Modelos evaluados	21
3.6.4	Selección del juez	21
3.6.5	Resultado de la selección	24
CAPÍTULO 4: VALIDACIÓN DE LA SOLUCIÓN		25
4.1	ENFOQUE DE LA VALIDACIÓN	25
4.2	MÉTRICAS TÉCNICAS	25
4.2.1	Exactitud del reconocimiento de voz (STT)	25
4.2.2	Latencia del <i>backend</i>	26
4.2.3	Latencia de captura	26
4.2.4	Pertinencia de las respuestas	26
4.3	PRUEBAS CON USUARIOS: ENTREVISTAS	26
4.3.1	Diseño	26
4.3.2	Instrumento: puntaje de satisfacción	27
4.3.3	Participantes	27
4.3.4	Sesiones	27
4.3.5	Caso 1: primer usuario (P1)	27
4.3.6	Casos 2 a 4: P2 (dos sesiones) y P3	27
4.4	ANÁLISIS DE PATRONES	28
4.5	RETROALIMENTACIÓN DE LOS USUARIOS	28
4.6	CONCLUSIÓN DE LA VALIDACIÓN	29
CAPÍTULO 5: CONCLUSIONES		30
ANEXOS		31
5.1	ENCUESTA SOBRE TECNOLOGÍAS PARA PERSONAS CIEGAS	31
5.1.1	Respuestas completas	32
5.2	CALIBRACIÓN DEL JUEZ LLM: HISTORIAL DE VERSIONES DEL <i>PROMPT</i>	40
REFERENCIAS BIBLIOGRÁFICAS		42

ÍNDICE DE FIGURAS

1	Arquitectura general del relator de pantalla.	15
---	---	----

ÍNDICE DE TABLAS

1	Clasificación de los lectores de pantalla más usados.	7
2	Comparación de las modalidades de interacción con el usuario.	17
3	Comparación de los entornos de inferencia probados.	19
4	Modelos VLM candidatos evaluados sobre el conjunto completo de preguntas.	22
5	Comparación de los modelos LLM probados como candidatos a juez, incluyendo el descarte preliminar y los tres candidatos evaluados formalmente.	23
6	Ficha técnica de la encuesta.	31
7	Respuestas completas de la encuesta, anonimizadas. Fuente: Elaboración propia.	32
8	Resultados de <i>exact match</i> por versión de <i>prompt</i> y modelo juez. Fuente: Elaboración propia.	40

INTRODUCCIÓN

Debe proporcionar a un lector los antecedentes suficientes para poder contextualizar en general la situación tratada, a través de una descripción breve del área de trabajo y del tema particular abordado, siendo bueno especificar la naturaleza y alcance del problema; así como describir el tipo de propuesta de solución que se realiza, esbozar la metodología a ser empleada e introducir a la estructura del documento mismo de la memoria.

En el fondo, que el lector al leer la Introducción pueda tener una síntesis de cómo fue desarrollada la memoria, a diferencia del Resumen dónde se explicita más qué se hizo, no cómo se hizo.

- Área: cruce de accesibilidad web e IA (visión / VLM).
- Brecha: los lectores de pantalla no interpretan contenido visual; oportunidad de una capa intermediaria.
- Solución: el relator de pantalla como complemento, no reemplazo del lector de pantalla.
- Encuesta: punto de partida que motiva y valida el enfoque.
- Hoja de ruta: encuesta ->prototipo ->*benchmark* ->validación en más detalle.

CAPÍTULO 1

DEFINICIÓN DEL PROBLEMA

1.1. PROBLEMÁTICA SOCIAL

Las personas con discapacidad visual enfrentan barreras persistentes al interactuar con entornos digitales. La Organización Mundial de la Salud reporta que al menos 2.200 millones de personas en el mundo tienen algún grado de limitación visual [World Health Organization, 2026], pero esa cifra agrupa mayoritariamente casos leves y corregibles con lentes. Al filtrar por los niveles de discapacidad severa y ceguera total, las estimaciones del *Global Burden of Disease* para el año 2020 son de 43,3 millones de personas ciegas y 295 millones con discapacidad visual moderada a severa, es decir, un orden de 338 millones de personas a nivel mundial [Bourne *et al.*, 2021].

Sin embargo, la magnitud de esta cifra no es el enfoque, sino la relevancia ética de integrar a un grupo de usuarios históricamente segregado del entorno tecnológico general. El problema de fondo no es que existan herramientas pensadas específicamente para personas ciegas, sino garantizar un acceso equitativo a ellas. En el caso de los lectores de pantalla, esa barrera económica es más acotada de lo que podría parecer: junto a alternativas de uso extendido a nivel profesional que requieren una licencia paga, como JAWS [Freedom Scientific, 2024], existen opciones de código abierto y sin costo, como NVDA, que su propio desarrollador describe como funcionalmente equivalentes [NV Access, 2024]. La barrera realmente transversal, independiente de si el lector de pantalla usado tiene costo o no, es la de aprendizaje: mientras que una persona sin discapacidad visual puede aprender a usar un computador de forma prácticamente autodidacta, dominar un lector de pantalla implica memorizar un número considerable de combinaciones de teclado —solo la guía rápida oficial de navegación web de JAWS, que cubre apenas una fracción de sus funciones totales, ya reúne cerca de 90 combinaciones distintas [Deque Systems, 2019]— y, con frecuencia, tomar cursos de capacitación formal —dictados por organizaciones especializadas o por el propio fabricante— antes de poder utilizarlo con fluidez. Evitar segregar o estigmatizar a un grupo de usuarios mediante soluciones separadas ha sido, de hecho, uno de los principios formales del diseño universal —el principio de uso equitativo— en los últimos 20 años [Mace y The Center for Universal Design, 1997].

La consecuencia de no resolver estas barreras es la exclusión en las dimensiones social, laboral y educativa: la imposibilidad de acceder de forma autónoma a contenido en línea limita la participación social cotidiana, el desempeño y las oportunidades laborales, y el acceso a instancias educativas. A modo de ejemplo, hoy no es posible enterarse por cuenta propia de qué muestra una fotografía que comparte un familiar o un amigo, ni completar sin ayuda una compra o un trámite en línea cuando de por medio se exige una verificación visual [WebAIM, 2024]. Tampoco es posible darse cuenta a tiempo de un aviso que aparece de forma inesperada en la pantalla, lo que puede significar perder una oportunidad o un plazo

importante. En el trabajo, muchas veces es necesario pedirle a un compañero que describa una cifra o una imagen antes de poder tomar una decisión. Algo similar ocurre en lo educativo, donde acceder a un material de estudio compartido como imagen deja a la persona a la espera de que alguien más se lo describa antes de poder seguir aprendiendo al mismo ritmo que el resto.

1.1.1. Antecedentes y encuesta propia

Se estableció que las personas ciegas enfrentan esta problemática social de forma particular al navegar la web. Sin embargo, al revisar el estado del arte no se encontraron estudios específicos que caracterizaran con suficiente detalle esa necesidad en este grupo, sobre todo porque ya existen soluciones tecnológicas que resuelven parte de esto, como por ejemplo los lectores de pantalla¹. Ante la ausencia de antecedentes que caracterizaran esta necesidad con suficiente detalle, se diseñó y aplicó una encuesta propia con el fin de validar la problemática empíricamente antes de proponer una solución.

En este contexto, se elaboró el instrumento “Encuesta sobre tecnologías para personas ciegas”, diseñado por el autor y validado previamente con el director de FUNDALURP (Fundación de Lucha contra la Retinitis Pigmentosa), organización dedicada a la rehabilitación de personas en situación de discapacidad visual. La encuesta se distribuyó mediante Google Forms por correo electrónico a personas asistidas por la fundación o vinculadas a ella, y se aplicó entre febrero y marzo de 2026, obteniéndose 9 respuestas. El detalle completo del instrumento —su estructura, tipos de pregunta y cómo se relacionan con la problemática— se presenta en el Anexo 5.1.

El perfil de los encuestados muestra como condición dominante la retinitis pigmentaria, con niveles de discapacidad que van desde severo hasta ceguera total, y un manejo computacional autoreportado mayoritariamente entre básico e intermedio. De los 9 encuestados, 7 (78 %) reportaron emplear un lector de pantalla; entre ellos, NVDA fue el más señalado como lector principal (4 de 7, 57 %), seguido de VoiceOver (2 de 7, 29 %) y TalkBack (1 de 7, 14 %). Ninguno reportó usar JAWS como lector principal, aunque más de la mitad de quienes usan un lector de pantalla (4 de 7) sí lo reconoció como una herramienta conocida.

Respecto de las dificultades reportadas al utilizar un lector de pantalla para navegar la web, en orden de más a menos seleccionada entre los 7 encuestados que usan uno, las más señaladas fueron: elementos estrictamente visuales que no pueden interpretarse (imágenes, gráficos); *pop-ups* o notificaciones que no se leen por falta de etiquetado *aria-label*; un orden de lectura ilógico que obliga a recorrer información irrelevante antes de llegar a la relevante; y los *captchas* visuales. Con una frecuencia levemente menor (4 de 7, 57 % cada una) les siguen los campos de escritura sin etiqueta, las imágenes informativas sin texto alternativo, y las instrucciones dadas en términos estrictamente visuales.

¹Programas que traducen el contenido de una página web a voz sintetizada o a un dispositivo braille; se profundiza en su funcionamiento en el Capítulo 2.

En cuanto a las funcionalidades identificadas como las más necesitadas, en orden según la proporción de encuestados que las seleccionó, se destacan dos empatadas en primer lugar (6 de 7, 86 % cada una): describir e interpretar imágenes, fotografías y gráficos; y la navegación por voz hacia un objetivo concreto. Le sigue, en un tercer lugar más distanciado (4 de 7, 57 %), obtener un resumen del contenido principal del sitio que se está visitando.

1.2. PROBLEMÁTICA TÉCNICA

1.2.1. Funcionamiento de los lectores de pantalla

Actualmente, las personas con discapacidad visual navegan la web apoyándose en lectores de pantalla [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024], programas como JAWS, NVDA o VoiceOver que traducen el contenido de una página a voz sintetizada o a un dispositivo braille —un dispositivo físico con celdas que forman y deshacen caracteres en relieve mediante pines accionados electromecánicamente, y que se conecta al computador para ir mostrando en tiempo real, al tacto, el contenido que el lector de pantalla va comunicando—. Para saber qué comunicar, un lector de pantalla no percibe la página como lo haría una persona con capacidad visual plena, sino que consulta la API de accesibilidad del sistema operativo [MDN Web Docs, 2024a], una interfaz que el propio navegador alimenta a partir del DOM (*Document Object Model*): la estructura de nodos que el navegador construye desde el código HTML de la página en su parte estática inicial, y que además tiene una dimensión programática, ya que JavaScript puede modificar ese DOM en tiempo de ejecución, agregando, quitando o alterando nodos después de la carga inicial. A partir de ese DOM, el navegador deriva un árbol de accesibilidad —una representación paralela que expone únicamente los roles, nombres accesibles, estados y relaciones relevantes para una tecnología asistiva— y se lo entrega al sistema operativo a través de esa misma API; es el sistema operativo quien, a partir de ahí, pone ese árbol a disposición de cualquier tecnología asistiva que lo consulte, incluido el lector de pantalla. Es a ese árbol al que el lector de pantalla accede para elaborar lo que finalmente comunica al usuario. También existe la posibilidad de que el usuario navegue por una página utilizando únicamente el teclado, lo cual es posible gracias al atributo `tabindex` y al orden del DOM. El recorrido secuencial mediante la tecla Tab lo controla el propio navegador para la navegación por teclado, en el orden del DOM o según el atributo `tabindex`, y no es exclusivo de los lectores de pantalla ni de los usuarios con discapacidad visual.

1.2.2. Dependencia de la semántica: ARIA y contenido dinámico

Buena parte de la información que un lector de pantalla puede comunicar depende de las etiquetas ARIA (*Accessible Rich Internet Applications*): un conjunto de atributos que se agregan a los elementos del DOM para declarar roles, estados y relaciones que el HTML sin mar-

cadoreos adicionales no permite describir por sí solo [MDN Web Docs, 2024b]. Por ejemplo, una notificación emergente puede marcarse como una región viva (`aria-live`) para que el lector la anuncie apenas aparece. Cuando un sitio no declara estas etiquetas, el lector de pantalla no tiene forma de identificar ese contenido dinámico, aunque técnicamente sí forme parte del DOM.

1.2.3. Límites del enfoque

El límite de este enfoque está, entonces, en que depende por completo de que la página declare una semántica correcta y suficiente para cada elemento —ya sea mediante etiquetas HTML semánticas (por ejemplo `<article>` o `<dialog>`) o mediante ARIA. Los lectores de pantalla no interpretan información estrictamente visual: no describen imágenes, gráficos ni *captchas*, y solo leen de forma confiable las notificaciones o *pop-ups* que fueron etiquetados correctamente por quien construyó el sitio. Tampoco comprenden la estructura espacial de la interfaz (qué elemento está dónde y con qué propósito) más allá del orden en que fueron declarados. El usuario recibe así una representación incompleta de la página, que depende de decisiones de accesibilidad ajenas a su control. Para una persona con visión, en cambio, el uso de la interfaz es evidente y directo: percibe de un vistazo la totalidad de la página y su disposición espacial, sin depender de que un tercero haya declarado correctamente su semántica.

1.2.4. Soluciones existentes y su insuficiencia

Respecto de la competencia y soluciones existentes, las alternativas actuales resultan insuficientes frente a estas brechas de accesibilidad [Campbell *et al.*, 2024]. Los lectores de pantalla, por diseño, no cubren la interpretación visual; y si bien existen soluciones basadas en IA para solicitar descripciones de imagen, estas no están integradas dentro del flujo de navegación con lector de pantalla ni fueron pensadas específicamente para personas ciegas, por lo que solo resuelven el problema de forma parcial y bajo ciertas condiciones.

1.3. OBJETIVOS Y ALCANCES

Considerando la relevancia de que las personas con discapacidad visual puedan acceder de forma equitativa a las tecnologías de la información, se plantean a continuación los objetivos que guían el desarrollo del asistente de accesibilidad web propuesto en esta memoria.

Objetivo general. Desarrollar una herramienta denominada relator de pantalla, que permita acceder a información descriptiva de un sitio web mediante interacciones de pregunta y respuesta en lenguaje natural, y que sea compatible con el lector de pantalla que el usuario ya utiliza, funcionando junto a él sin reemplazarlo ni interferir en su operación.

Objetivos específicos.

- (a) Implementar un prototipo funcional del relator de pantalla capaz de interpretar información visual y entregar descripciones auditivas.
- (b) Ajustar el comportamiento del sistema, mediante el afinamiento de sus instrucciones, para adaptarse de mejor forma a las necesidades específicas del usuario.
- (c) Empaquetar la solución para una disponibilización básica entre usuarios.
- (d) Validar la solución mediante pruebas con usuarios reales.

Alcances. Se espera que las personas ciegas puedan interactuar de forma más directa con el computador para acceder a información que el lector de pantalla actualmente no les puede proveer de forma nativa, facilitando el acceso a información visual de la pantalla mediante interacciones fáciles de utilizar para este grupo de usuarios.

CAPÍTULO 2

MARCO CONCEPTUAL

2.1. LECTORES DE PANTALLA Y EL DOM

Un lector de pantalla es un software de tecnología asistiva que convierte el contenido de una interfaz digital en voz sintetizada o en salida braille, permitiendo al usuario acceder a esa información sin necesidad de verla [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024]. La salida braille se entrega mediante una línea braille refrescable: un dispositivo de hardware, conectado al computador, con celdas que forman y deshacen caracteres en relieve mediante pines accionados electromecánicamente, y que va mostrando al tacto y en tiempo real el contenido que el lector de pantalla comunica.

Los lectores de pantalla no son una categoría homogénea ni universal: existen embebidos en el propio sistema operativo, como Windows Narrator o VoiceOver en macOS/iOS, y otros que se instalan por separado, como JAWS o NVDA; dentro de estos últimos hay opciones de código abierto y sin costo (NVDA) y opciones pagas orientadas al uso profesional (JAWS); y hay tanto lectores de escritorio como lectores para dispositivos móviles, como TalkBack en Android. Qué lector de pantalla resulta más adecuado depende, entonces, del sistema operativo, el dispositivo y el presupuesto disponible, y no existe uno que cubra todos los contextos de uso por igual.

Tabla 1: Clasificación de los lectores de pantalla más usados.
Fuente: Elaboración propia,
a partir de la Encuesta de Usuarios de Lectores de Pantalla #10 de WebAIM [WebAIM, 2024].

Lector	Tipo	Licencia	Plataforma	Uso reportado
JAWS	Instalado	Pago	Escritorio	40,5 % como lector principal
NVDA	Instalado	Código abierto, sin costo	Escritorio	37,7 % como lector principal
VoiceOver	Embebido (macOS/iOS)	Incluido con el sistema operativo	Escritorio y móvil	9,7 % como lector principal de escritorio; 70,6 % de uso reportado en móviles
Narrator	Embebido (Windows)	Incluido con el sistema operativo	Escritorio	0,7 % como lector principal
TalkBack	Embebido (Android)	Incluido con el sistema operativo	Móvil	34,7 % de uso reportado en móviles

Las cifras de escritorio corresponden a la pregunta por el lector de pantalla **principal**, mientras que las cifras de uso en dispositivos móviles corresponden a una pregunta distinta por los lectores que el encuestado usa **con frecuencia**, sin exclusividad entre ellos [WebAIM, 2024]; por eso no son directamente comparables entre sí ni suman 100 %.

El DOM (*Document Object Model*) es la representación estructurada del documento web: una jerarquía de nodos (elementos, atributos, texto) que el navegador construye a partir del código HTML de la página en su parte estática inicial, y sobre la que se puede consultar y operar mediante programación [WHATWG, 2026]. Esa segunda dimensión, programática, es la que aporta JavaScript: al ejecutarse, puede agregar, quitar o modificar nodos del DOM ya construido, de modo que lo que el navegador termina exponiendo no depende solo del HTML servido inicialmente, sino también de los cambios que JavaScript introduce después. El navegador deriva del DOM resultante un árbol de accesibilidad, una representación paralela y más acotada que expone únicamente los roles, nombres, estados y relaciones relevantes para una tecnología asistiva, a través de la API de accesibilidad del sistema operativo (por ejemplo, UI Automation en Windows o AXAPI en macOS). Es esa API la que el lector de pantalla consulta para construir lo que finalmente comunica al usuario.

La capacidad del navegador de construir un árbol de accesibilidad completo depende de que la página declare correctamente su semántica, algo que el HTML sin marcadores adicionales no siempre garantiza. El W3C define para ello dos estándares complementarios: las Pautas de Accesibilidad para el Contenido Web (WCAG), que establecen los criterios que debe cumplir un sitio para considerarse accesible [Campbell *et al.*, 2024], y WAI-ARIA, un conjunto de atributos (roles, estados y propiedades `aria-*`) que permite declarar la semántica de elementos que no la expresan por sí solos —por ejemplo, marcar una notificación emergente como una región viva (`aria-live`) para que se anuncie apenas aparece— [Nurthen *et al.*, 2023]. El atributo `alt`, por su parte, es el mecanismo nativo para asociar un texto alternativo a una imagen —y existe desde antes que los lectores de pantalla, no exclusivamente para ellos—; sin él, una imagen no tiene ninguna representación textual que el lector de pantalla pueda comunicar [World Wide Web Consortium (W3C), Web Accessibility Initiative, 2014].

El lector de pantalla permite al usuario recorrer **secuencialmente** el contenido de la página mediante la tecla Tab, uno tras otro: no solo los elementos interactivos como campos de texto o enlaces, sino también contenido no interactuable como párrafos de texto. Ese orden de recorrido no es estrictamente el orden en que los elementos fueron declarados en el HTML: el atributo `tabindex` permite alterarlo, adelantando o postergando elementos respecto de su posición original en el documento. Además de ese recorrido, el lector de pantalla ofrece ayudas de navegación que no existen sin él: una de las más usadas es el listado de enlaces, que presenta todos los enlaces de la página en una lista y permite saltar directamente al que se necesite.

La eficacia de este recorrido secuencial depende de la familiaridad previa del usuario con la estructura de la página: sobre un sitio ya explorado, permite ubicar rápidamente los elementos buscados; frente a un sitio nuevo, requiere recorrerlo y estudiarlo varias veces antes de poder anticipar su estructura.

2.2. EXTENSIONES DE NAVEGADOR

Una extensión es un módulo de software que un usuario instala sobre su navegador para modificar o ampliar su comportamiento —por ejemplo, para bloquear anuncios, gestionar contraseñas o capturar información de una página—, sin ser una aplicación independiente ni una función nativa del propio navegador [MDN Web Docs, 2024c].

Google Chrome fue el primer navegador en incorporar un sistema de extensiones basado enteramente en HTML, CSS y JavaScript, habilitado en su canal de desarrollo en 2009 y lanzado de forma pública junto a Chrome 4.0 en enero de 2010 [Chromium Blog, 2010]. A partir de esa base, Mozilla adoptó en 2015 una API compatible para su navegador Mozilla Firefox, bautizada WebExtensions, que buscaba convertirse en un estándar común entre navegadores; en 2021 se llegó a formar un grupo comunitario del W3C específicamente para intentar formalizar esa base común entre distintos fabricantes [World Wide Web Consortium (W3C), 2021], pero ese intento de estandarización no llegó a concretarse. Pese a ello, la popularidad de Chrome llevó a que el resto de los navegadores basados en su motor Chromium (Edge, Opera, Brave, entre otros) convergieran hacia una API de extensiones muy similar [MDN Web Docs, 2024c].

Hoy existen, en términos generales, dos grandes familias de navegador incompatibles entre sí: los basados en Chromium, que agrupan a la gran mayoría, y Mozilla Firefox, basado en el motor Gecko y heredero del antiguo navegador Netscape. Aparte de estas dos familias existen otros navegadores, como los propietarios de ciertos fabricantes de dispositivos móviles. Un caso aparte es Safari que utiliza su propio motor, WebKit. Independiente de que un navegador soporte extensiones, estas solo son compatibles con el navegador para el que fueron diseñadas: el código y el empaquetado de una extensión para Chrome no son directamente instalables en Firefox ni viceversa, aún cuando ambas compartan la mayoría de las llamadas de su API.

A nivel de capacidades, una extensión se ejecuta bajo un modelo de permisos explícitos declarados en un archivo de manifiesto: solo puede acceder a las páginas, pestañas o funciones del navegador para las que fue autorizada, y se organiza mediante scripts de contenido —que se ejecutan en el contexto de la página visitada— y un script de fondo, que coordina la lógica de la extensión y sus comunicaciones con servicios externos. Ese modelo de permisos delimita tanto lo que la extensión puede automatizar dentro del navegador como lo que queda fuera de su alcance sin intervención directa del usuario o del sistema operativo.

2.3. RECONOCIMIENTO Y SÍNTESIS DE VOZ (STT Y TTS)

El reconocimiento automático del habla (*automatic speech recognition*, ASR; también conocido como *speech to text*, STT) es el proceso mediante el cual un sistema decodifica una señal de audio con voz humana y la transcribe a texto, típicamente combinando un modelo acús-

tico —que traduce la señal a una secuencia probable de fonemas— y un modelo de lenguaje —que determina, a partir de esos fonemas, las palabras más probables— [IBM, 2024e].

La síntesis de voz (*text to speech*, TTS) es el proceso inverso: toma una entrada textual y la convierte en audio hablado. Esa entrada no siempre es texto plano: estándares como SSML (*Speech Synthesis Markup Language*), del W3C, permiten anotar el texto con marcadores de énfasis, acento, velocidad o pausas, para un control más fino de cómo se pronuncia [Burnett y Shuang, 2010]. A nivel de técnica, la síntesis de voz se ha resuelto históricamente de más de una forma: por concatenación de fragmentos de voz pregrabada, por síntesis paramétrica a partir de un modelo estadístico de la voz, y, más recientemente, mediante modelos neuronales generativos que producen la forma de onda directamente a partir de datos de entrenamiento [Tan *et al.*, 2021].

Lectores de pantalla como JAWS o NVDA no necesariamente implementan su propio motor de síntesis: en general, delegan la conversión final de texto a voz a un motor TTS del sistema operativo o a una voz de terceros instalada por el usuario, y lo que le corresponde al lector de pantalla es determinar qué texto de la interfaz enviar a ese motor.

2.4. IA MULTIMODAL: MODELOS DE LENGUAJE VISUAL (VLM)

2.4.1. IA generativa de lenguaje

Los modelos de lenguaje generativos son un tipo de inteligencia artificial capaz de producir texto nuevo a partir de patrones aprendidos de grandes volúmenes de datos, en lugar de limitarse a clasificar o predecir una etiqueta sobre texto ya existente [IBM, 2024a]. Esa generación se implementa técnicamente como una predicción: un modelo autoregresivo —que genera cada nueva unidad de texto a partir de lo que él mismo generó previamente—, como los que se basan en la arquitectura Transformer —una arquitectura de red neuronal que pondera mediante mecanismos de atención qué partes de la entrada son más relevantes para cada predicción—, no produce una respuesta completa de una sola vez: predice, *token* a *token* —el *token* es la unidad mínima de texto en que el modelo divide el lenguaje, por ejemplo una palabra o parte de ella—, cuál es la unidad de texto más probable dado todo lo que ha recibido y generado hasta ese momento, y repite ese proceso hasta completar la respuesta [Vaswani *et al.*, 2017]. Es decir, “generar” texto y “predecir la palabra siguiente” no son dos capacidades distintas, sino una relación de mecanismo a resultado: el modelo predice de forma iterativa, *token* a *token*, y esa cadena de predicciones es percibida por quien lo usa como la generación de una respuesta nueva.

2.4.2. IA multimodal

La IA multimodal corresponde a modelos de IA capaces de procesar y/o producir información de más de un tipo de dato —texto, imágenes, audio o video— de forma conjunta, en lugar de limitarse a un único tipo de entrada o salida [Wu *et al.*, 2023]. Dentro de esa familia se encuentran los modelos de lenguaje visual (VLM, *vision language model*), especializados en la interacción entre texto e imágenes: reciben como entrada una combinación de imagen(es) y texto, y devuelven una salida en texto, apoyándose en una representación interna que alinea los conceptos visuales de la imagen con los conceptos del lenguaje [Wu *et al.*, 2023].

2.4.3. Entrenamiento

El entrenamiento es el proceso mediante el cual un modelo de IA aprende a reconocer patrones y correlaciones para optimizar su desempeño en tareas similares a las de su uso previsto [IBM, 2024c]. Ese aprendizaje se realiza sobre un *dataset*: una colección de datos de ejemplo relevantes para la tarea que el modelo debe resolver —por ejemplo, pares de pregunta y respuesta, o imágenes junto a su descripción—; cuanto más se asemeje ese conjunto a los casos reales que el modelo enfrentará después, mejor será su desempeño sobre datos nuevos [IBM, 2024f].

Un entrenamiento puede fallar en dos direcciones opuestas. Por sobreajuste (*overfitting*), el modelo se ajusta tan estrechamente a los ejemplos de entrenamiento que termina memorizándolos en lugar de generalizar, logrando buenos resultados solo sobre datos ya vistos; por subajuste (*underfitting*), el modelo es demasiado simple, o el entrenamiento demasiado breve, para siquiera capturar los patrones del propio conjunto de entrenamiento [IBM, 2024d]. Un entrenamiento adecuado, en cambio, encuentra un punto intermedio entre ambos extremos, que le permite generalizar razonablemente bien a datos nuevos de una naturaleza similar a los ya vistos.

De ahí se desprende una limitación central de cualquier modelo de IA: no puede responder de forma confiable sobre información que su *dataset* de entrenamiento no consideró, o que consideró en una cantidad insuficiente para generalizar ese patrón sin sesgo. Cuando un tipo de caso está subrepresentado en el *dataset*, el modelo tiende a fallar o a generalizar incorrectamente para ese caso a partir de lo aprendido sobre los casos mayoritarios [IBM, 2024b]. En términos simples, un modelo entrega mejores resultados sobre aquello que aparece con frecuencia en su *dataset* de entrenamiento, y resultados progresivamente peores sobre aquello que aparece con poca frecuencia o no aparece en él.

2.4.4. Inferencia

La inferencia es el acto de usar un modelo de IA ya entrenado para obtener una salida a partir de una entrada nueva [NVIDIA, 2024]. Ejecutar esa inferencia sobre un modelo de gran tamaño es, en sí mismo, un desafío computacional: un equipo de escritorio promedio no puede hacerlo, o al menos no en tiempos que permitan un uso práctico, por lo que habitualmente se recurre a hardware específico —típicamente GPU— para hacerla eficiente. Frente a esto existen dos caminos habituales, ambos con inconvenientes: los servicios de IA en línea corren sobre datacenters de gran escala y cobran por consulta; además, procesan los datos del usuario en infraestructura de terceros, sin garantías claras sobre su tratamiento.

En este contexto aparece el concepto de latencia: el tiempo de espera entre que se envía una consulta y se recibe la respuesta. Es un factor crítico para la experiencia de uso, ya que una espera prolongada puede interrumpir el flujo de la interacción del usuario, incluso cuando el contenido de la respuesta es correcto.

2.5. EVALUACIÓN AUTOMATIZADA DE VLMs

Existe una gran oferta de modelos VLM *open source*, y distintos modelos entregan resultados diferentes para las mismas consultas, por lo que elegir el más pertinente para una necesidad determinada no es trivial. Hacerlo de forma manual —tomar un VLM, ingresarle N preguntas y evaluar cada una de esas N respuestas— no escala: para que el resultado sea confiable se necesita un N grande, y probar varios modelos vuelve la evaluación manual una tarea repetitiva e inviable. De ahí la necesidad, documentada en la literatura, de automatizar este proceso mediante técnicas de *benchmarking* sistemático.

2.5.1. Rúbrica cualitativa

Una rúbrica es un instrumento de evaluación cualitativa que describe, para cada nivel de desempeño posible, los criterios concretos que una respuesta debe cumplir para alcanzarlo; se construye de forma manual antes de la evaluación y permite calificar de forma consistente incluso cuando la respuesta es abierta y no tiene una única forma correcta —por ejemplo, al calificar la respuesta de un estudiante en un examen de argumentación, donde una rúbrica puede distinguir niveles que van desde una postura sin fundamento hasta un argumento completo y bien sustentado. La evaluación de un VLM enfrenta el mismo problema de fondo: la respuesta a una consulta sobre una imagen tampoco tiene una única forma correcta, por lo que una rúbrica permite distinguir niveles de calidad —por ejemplo, si identifica los elementos relevantes de la imagen, si omite información importante, o si incluye contenido que no está presente en ella— en lugar de exigir una coincidencia exacta con una respuesta de referencia.

2.5.2. *LLM-as-a-judge*

“*LLM-as-a-judge*” es un paradigma de evaluación en el que un LLM, en lugar de un evaluador humano, cumple el rol de juez: se le entrega la respuesta que se quiere evaluar junto con los criterios de evaluación —por ejemplo, una rúbrica—, y el modelo determina si esos criterios están presentes o ausentes para asignar un puntaje de forma cuantitativa [Zheng *et al.*, 2023].

2.5.3. *LLM-as-a-judge-evaluation*

Para confirmar que un LLM evaluador funciona bien, en este trabajo se introduce el concepto de “*LLM-as-a-judge-evaluation*”: un humano califica de forma independiente las mismas respuestas que evalúa el LLM, y luego se contrasta cuánto se acercan los puntajes de uno y otro. Es una segunda capa de evaluación, destinada a comprobar que el criterio con el que el LLM juzga esas respuestas está bien calibrado.

Con la *LLM-as-a-judge-evaluation* se cierra el ciclo: el humano evalúa las respuestas, el LLM las evalúa también, y la comparación entre ambos permite validar al LLM evaluador a partir del criterio humano.

2.5.4. Herramientas estandarizadas de evaluación

Respecto de los sistemas estandarizados de evaluación específica para VLMs, ya existen herramientas y métricas destinadas a esta tarea. Un ejemplo es VLMEvalKit (open-compass): un *toolkit open source* de evaluación de modelos multimodales que unifica la evaluación de una gran cantidad de modelos sobre decenas de *benchmarks* con configuración mínima, automatizando la preparación de datos, la inferencia y el cálculo de métricas, e incluso usando un LLM como extractor de respuestas cuando la comparación exacta falla [Duan *et al.*, 2024]. Sin embargo, ninguna de estas herramientas está diseñada específicamente para la lectura de sitios web en el contexto de la discapacidad visual, que es el foco de este trabajo.

CAPÍTULO 3

PROPUESTA DE SOLUCIÓN

Se plantea desarrollar un programa que, mediante un atajo de teclado, permita al usuario indicar qué necesita saber de la pantalla y reciba de vuelta una respuesta generada por el sistema. Ese comando lo recibe una extensión de navegador, que envía una captura de la pantalla junto con la pregunta a un servidor de inferencia; este la procesa y retorna una respuesta en texto que, finalmente, es transformada a audio por el lector de pantalla del propio usuario.

Las secciones siguientes describen, en ese orden, la arquitectura general del sistema, la extensión de navegador, el *backend* que procesa las consultas, la infraestructura sobre la que corre el modelo, y el proceso de selección del modelo VLM utilizado.

3.1. ARQUITECTURA GENERAL DEL RELATOR DE PANTALLA

La arquitectura propuesta se compone de una extensión de navegador y un *backend* que coordinan la captura de la pantalla, el envío de la consulta al modelo, y la entrega de la respuesta. Esa respuesta, sin embargo, no llega al usuario mediante un mecanismo de audio propio del sistema: se apoya en la misma vía por la que el lector de pantalla ya comunica cualquier otro contenido de una página, es decir, en el DOM y en el árbol de accesibilidad que el navegador expone a través de la API de accesibilidad del sistema operativo.

El flujo inicia en la extensión de navegador, instalada sobre el navegador del propio usuario. Al presionar el atajo de teclado configurado, la extensión captura la pantalla —el *viewport* visible o la página completa, según el atajo ejecutado— y recoge la pregunta del usuario, ya sea escrita o dictada por voz. Con ambos datos arma una solicitud HTTP (POST `/api/ask`) hacia el *backend*.

El *backend*, o capa controladora, recibe esa solicitud y actúa como intermediario hacia el servidor de inferencia: reenvía la imagen y la pregunta mediante otra solicitud HTTP (POST `/generate`), agregando una llave de autenticación (`api_key`) para acceder a dicho servidor.

El servidor de inferencia aloja el modelo VLM y es, en la práctica, el único punto del sistema donde ocurre cómputo pesado sobre GPU. Procesa la imagen y la pregunta recibidas, y devuelve un JSON con el texto generado y algunas métricas de la ejecución.

Esa respuesta vuelve entonces al *backend*, que la reenvía a la extensión como un JSON con el estado de la solicitud y el texto de respuesta (`response_text`).

Al recibir ese texto, la extensión no lo transforma directamente a audio ni implementa nin-

gún mecanismo de voz propio: lo inserta en el DOM de la página como una región viva (aria-live), tal como lo haría cualquier notificación dinámica del propio sitio.

Ese cambio en el DOM es recogido por el navegador, que deriva de él un árbol de accesibilidad actualizado y lo expone a través de la API de accesibilidad del sistema operativo.

Finalmente, es el propio lector de pantalla del usuario —externo al sistema, sea cual sea el que tenga instalado— quien consulta esa API y anuncia el contenido de la región viva, tal como anunciaría cualquier otro cambio dinámico de una página web.

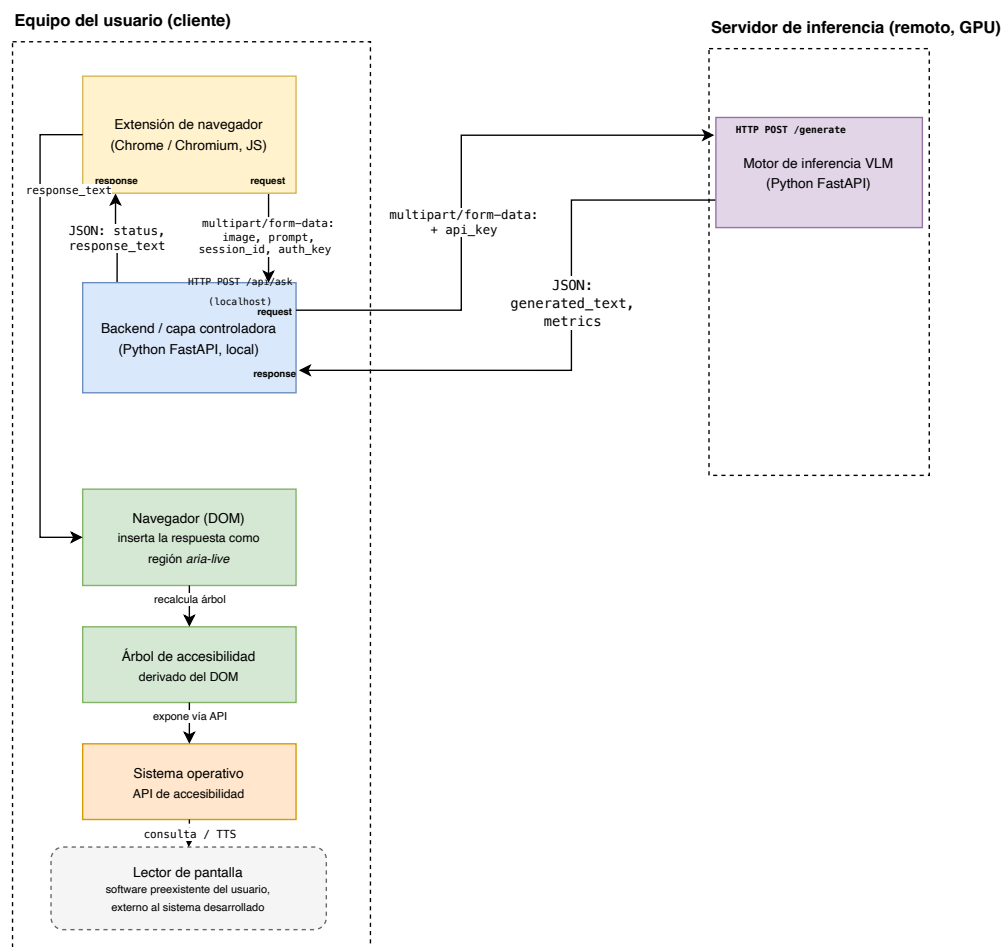


Figura 1: Arquitectura general del relator de pantalla.

Fuente: Elaboración propia.

El flujo completo, desde que se formula la pregunta hasta que se obtiene la respuesta, queda entonces de la siguiente forma:

1. El usuario presiona el atajo de teclado y formula su pregunta —por texto o por voz— a la extensión de navegador.

2. La extensión captura la pantalla y envía la captura junto con la pregunta al *backend* (HTTP POST /api/ask).
3. El *backend* reenvía la captura y la pregunta al servidor de inferencia VLM (HTTP POST /generate).
4. El servidor de inferencia procesa la consulta y devuelve el texto generado al *backend*.
5. El *backend* devuelve ese texto a la extensión.
6. La extensión inserta el texto de respuesta en el DOM de la página, como una región viva (aria-live).
7. El navegador deriva del DOM actualizado un árbol de accesibilidad y lo expone a través de la API de accesibilidad del sistema operativo.
8. El lector de pantalla del usuario consulta esa API, detecta la región viva y la anuncia, completando así el ciclo iniciado por la pregunta.

3.2. EXTENSIÓN DE NAVEGADOR

Se desarrolló una extensión diseñada para navegadores basados en Chromium, que captura el *viewport* visible o la página completa mediante atajos de teclado configurables por el usuario.

3.2.1. *Scroll & stitch*

Para la captura de página completa se aplica una lógica de *scroll & stitch*: se congelan los elementos dinámicos del sitio y se aplica un desplazamiento (*scroll*) continuo a lo largo de la página, tomando una captura parcial en cada tramo recorrido y uniéndolas luego (*stitch*) en una sola imagen. Esto entrega la perspectiva completa y actual del sitio, según lo que requiera la consulta del usuario.

3.2.2. Formato de salida de la imagen

El output de las imágenes puede entregarse directamente en un formato de imagen (por ejemplo .jpg o .png) o convertirse a base64 para enviarlo más fácilmente por HTTP al *backend*. En cualquiera de los dos casos, el *backend* la recibe como parte de una solicitud multipart/form-data enviada mediante HTTP POST, junto con la pregunta, el identificador de sesión y la llave de autenticación (ver Figura 1).

3.3. INTERACCIÓN CON EL USUARIO

Además de capturar la pantalla, la extensión es responsable de la modalidad de interacción con el usuario, es decir, de si la consulta se hace por texto o por voz. Esta responsabilidad es exclusiva de la extensión: es irrelevante para el *backend* si la pregunta llegó escrita o hablada, dado que en ambos casos recibe únicamente texto.

3.3.1. Interacción sin síntesis de voz

En su forma más esencial, la extensión muestra un cuadro de texto donde el usuario escribe la pregunta, la envía al *backend*, y recibe la respuesta también como texto, mostrada en un *pop-up*. No hay reconocimiento ni síntesis de voz involucrados en ningún punto de esta interacción: tanto la pregunta como la respuesta quedan sobre la pantalla en forma de texto. Al no depender de ningún servicio externo de voz, es también la modalidad más simple de mantener y depurar.

3.3.2. Interacción con síntesis de voz

En su forma completa, la extensión incorpora reconocimiento de voz para la pregunta — mediante la Web Speech API de Google, que procesa el audio localmente o en la nube—, y la respuesta se comunica en voz alta a través del lector de pantalla del usuario. El usuario puede, sin embargo, optar por escribir la pregunta en lugar de decirla en voz alta y aun así recibir la respuesta hablada; esta flexibilidad importa para quienes prefieren escribir sus consultas, ya que dejar de escribir para hablarle al computador a mitad de una sesión puede romper el flujo habitual de trabajo del usuario.

Tabla 2: Comparación de las modalidades de interacción con el usuario.

Fuente: Elaboración propia.

Modalidad	Entrada	Salida
Sin síntesis de voz	Texto escrito	Texto, mostrado en un <i>pop-up</i>
Con síntesis de voz	Voz (reconocimiento por STT) o texto escrito, a elección del usuario	Voz, mediante el lector de pantalla del usuario

3.4. BACKEND DEL SISTEMA

El *backend* está implementado en Python con FastAPI. Su responsabilidad se limita a orquestar la comunicación entre la extensión y el servidor de inferencia, sin ejecutar cómputo de IA por sí mismo.

Esta separación en dos componentes —en lugar de que la extensión llame directamente al servidor de inferencia— responde ante todo a una razón de seguridad: la llave de autenticación (`api_key`) que da acceso a dicho servidor se mantiene únicamente en el *backend*, un entorno que el usuario no puede inspeccionar. Si esa llave se incluyera en el código de la extensión, cualquier persona con acceso al navegador podría extraerla y utilizarla sin autorización, dado que el código de una extensión es, en la práctica, público para quien la instala.

Esa misma separación desacopla, además, a la extensión de los detalles concretos del servidor de inferencia —su dirección, su disponibilidad, o el modelo VLM que corre en un momento dado—, de modo que ese servidor puede reemplazarse, reconfigurarse o escalar sin requerir cambios en la extensión ya distribuida a los usuarios.

3.5. INFRAESTRUCTURA Y SERVIDOR ALBERT

La infraestructura de ejecución del proyecto contempla dos partes: los equipos de los usuarios y el servidor de inferencia con GPU.

Del lado del usuario, basta un equipo con un navegador basado en Chromium; el sistema operativo puede ser Windows, macOS, Linux o cualquiera compatible con dichos navegadores.

Para la inferencia se utilizó “*albert*”, una máquina del laboratorio de *data science* que cuenta con una tarjeta gráfica NVIDIA A40 con 48 GB de VRAM, capaz de correr modelos VLM de alta cantidad de parámetros. Se accede a *albert* vía *jump host* desde el *backend* para enviar las consultas mediante conexión SSH, proceso que se automatiza a través de un túnel SSH.

3.5.1. Decisión de infraestructura

Al determinar la arquitectura de inferencia existían tres formas de resolverla: usar una API o una IA ya empaquetada de un tercero, levantar un servidor propio remoto (en la nube), o utilizar un servidor propio local. Se optó por un servidor propio, ya que permite controlar tanto el modelo utilizado como la infraestructura sobre la que corre, sin depender de las condiciones de un tercero ni de los costos por consulta de un servicio en línea.

Antes de llegar al servidor local dedicado, se evaluaron progresivamente otras tres alternativas para ejecutar un modelo VLM acotado. Las cuatro pruebas se ejecutaron sobre la misma consulta —preguntar por el contenido y la estructura de la página principal de la UTFSM— repetida 5 veces sobre cada entorno, para promediar el tiempo de respuesta (ver Tabla 3).

El tiempo de respuesta deseado para estas pruebas se fijó en no más de 10 segundos por consulta: ese es, según la literatura de interacción humano-computador, el límite bajo el cual un usuario se mantiene atento al diálogo con el sistema sin distraerse con otra tarea, en

particular cuando percibe que la información solicitada es valiosa [Nielsen, 1993]. Este es el concepto de latencia (tiempo de espera por consulta) introducido en el Capítulo 2, aplicado aquí como requisito operativo concreto del sistema.

La primera prueba se realizó en el equipo de desarrollo local, apoyándose únicamente en su CPU, ya que esta no contaba con una GPU compatible con los modelos evaluados. El promedio resultante fue de 34 minutos por consulta, un tiempo muy por sobre lo utilizable para una interacción en tiempo real.

Para descartar que ese resultado se debiera a limitaciones propias del equipo de desarrollo, se repitió la misma prueba en una instancia en la nube con una CPU más potente. El promedio bajó a 12 minutos por consulta: una mejora respecto del equipo local, pero aún claramente insuficiente.

Se probó entonces con una sesión de Google Colaboratory, esta vez aprovechando una GPU NVIDIA T4. El promedio bajó drásticamente a 1 minuto por consulta, confirmando que la variable relevante era el uso de GPU y no las mejoras incrementales de CPU.

Finalmente, al tener acceso a *albert* y su GPU NVIDIA A40, el promedio se redujo a aproximadamente 10 segundos por consulta.

Tabla 3: Comparación de los entornos de inferencia probados.

Fuente: Elaboración propia, promedio de 5 ejecuciones de la misma consulta por entorno.

Entorno	Hardware de ejecución	Especificaciones	RAM	Tiempo promedio
Equipo de desarrollo local	CPU	AMD Ryzen 5 2600 (6 núcleos / 12 hilos, 3,4–3,9 GHz, arquitectura Zen+)	16 GB	34 min
AWS EC2 g4dn.xlarge	CPU	Intel Xeon Platinum 8259CL “Cascade Lake” (4 vCPU, 2,5 GHz)	16 GiB	12 min
Google Colaboratory	GPU	NVIDIA T4, 16 GB VRAM	~12 GB (asignada dinámicamente, nivel gratuito)	1 min
<i>albert</i> (laboratorio de <i>data science</i> , DI)	GPU	NVIDIA A40, 48 GB VRAM	Por confirmar	10 s

El resultado de *albert* (~10 s) cumple el objetivo de latencia establecido previamente, muy por debajo de las alternativas basadas en CPU y superando también a Google Colaboratory. Bajo ese criterio de latencia —sumado a razones de costo, al evitar el pago recurrente por

un servicio de nube externo, y de seguridad de la información, al mantener las capturas de pantalla y las consultas de los usuarios dentro de infraestructura propia sin depender de terceros— se eligió a *albert* como servidor de inferencia definitivo para el proyecto.

3.6. BENCHMARK DE MODELOS VLM

Se realizó un procedimiento de *benchmarking* para seleccionar el VLM más adecuado para la tarea de analizar sitios web con perspectiva de accesibilidad, evaluando sobre sitios reales.

3.6.1. Sitios y preguntas

Se seleccionó un conjunto de 10 sitios web en 5 categorías:

- Instituciones educativas: USM, UCH.
- Organizaciones estatales: SII, SENADIS.
- Sitios de *e-commerce*: Falabella, Mercado Libre.
- Venta de entradas a eventos: Ticketmaster, Ticketplus.
- Foros y comunidades: Reddit, 9gag.

Para cada categoría se escribieron 3 preguntas generales, aplicables a todos los sitios que la componen, y para cada sitio se escribieron además 3 preguntas específicas, sumando un total de 6 preguntas por sitio. Considerando los 10 sitios del conjunto, esto da como resultado 60 preguntas evaluadas por cada modelo.

3.6.2. Rúbrica de evaluación

Se escribieron rúbricas con puntajes del 0 al 3 según qué tanto se acerca la respuesta a la de un ser humano. La rúbrica prioriza que el dato, elemento o texto puntual solicitado sea correcto y preciso, más que la redacción con la que se presente el resto de la respuesta: si la pregunta exige extraer un texto específico del sitio (por ejemplo, el rótulo exacto de un botón), ese texto debe reproducirse tal cual aparece; para el resto de los casos, basta con que el valor o elemento identificado sea el correcto, aunque se describa con palabras distintas. Un lenguaje condicional o ambiguo sobre la información principal solicitada —por ejemplo, señalar que algo “probablemente” se encuentra en cierto lugar— se penaliza como si no se hubiera identificado en absoluto. La no verbosidad es otro criterio aplicado: no se mejora el puntaje de una respuesta solo por ser extensa. Los cuatro niveles son:

- **0 - Alucinación o falsedad.** Se selecciona cuando la respuesta alucina contenido que no existe en ningún lugar de la captura, entrega información falsa respecto de lo que esta efectivamente muestra, o se limita a responder a partir del conocimiento previo o general del modelo en lugar de referirse a lo que la imagen realmente contiene, sin importar cuán coherente o segura suene. Por ejemplo, describir un elemento, un precio o una sección típica de ese tipo de sitio sin que la imagen lo confirme en absoluto.
- **1 - Parcial.** Se selecciona cuando la respuesta sí ubica correctamente, dentro de la propia captura, el elemento o la sección donde se encuentra la información solicitada, pero no logra extraer el texto o el detalle exacto que contiene. Por ejemplo, señalar acertadamente en qué parte de la página se ubica un botón o un bloque de texto, sin poder transcribir su contenido literal.
- **2 - Correcta pero incompleta.** Se selecciona cuando la respuesta identifica correctamente la información pedida, pero omite parte de ella o la generaliza en lugar de extraerla en su totalidad. Por ejemplo, mencionar que existe un listado de opciones u elementos relevantes sin enumerarlos todos.
- **3 - Exacta.** Se selecciona cuando la respuesta extrae la información pedida de forma literal y completa, sin agregar ni omitir contenido. Por ejemplo, citar textualmente una cifra o un texto exactamente como aparece en el sitio, sin parafrasear ni omitir parte de él.

3.6.3. Modelos evaluados

Antes de iniciar este proceso se probaron informalmente otros dos modelos, moondream2 e InternVL2-2B: el primero respondía con rapidez, pero de forma muy básica, y el segundo, si bien era más lento, intentaba entregar respuestas más elaboradas, aunque alucinaba con frecuencia. Al ver estos resultados, se decidió comenzar el proceso de *benchmark*, evaluando 5 modelos VLM candidatos sobre el conjunto completo de 60 preguntas, en orden creciente de parámetros y de uso de GPU, usando en esta etapa temprana gemma-3-12b como juez automatizado (ver Tabla 4).

Qwen3-VL-8B obtuvo el mejor resultado del grupo y fue el modelo seleccionado para el prototipo del relator de pantalla.

3.6.4. Selección del juez

Antes de definir los candidatos finales a juez, se probaron preliminarmente otros seis modelos —Llama 3.1 8B, Ministral 8B, Phi-4-mini, Qwen3-14B (con y sin razonamiento) y Qwen3-32B— haciéndolos evaluar el conjunto completo de preguntas con una versión temprana y no iterada del *prompt*. Sus puntajes fueron consistentemente bajos, entre 6 % y 56 % sobre

Tabla 4: Modelos VLM candidatos evaluados sobre el conjunto completo de preguntas.

Fuente: Elaboración propia.

Modelo	VRAM aprox.	Puntaje (juez gemma-3-12b)
moondream2	~4 GB	N/A – prueba preliminar, sin evaluación formal
InternVL2-2B	~5 GB	N/A – prueba preliminar, sin evaluación formal
InternVL3-2B	~4 GB	101/180 (56 %)
InternVL3-14B	~33 GB	118/180 (66 %)
MiniCPM-V-4.5	~17 GB	131/180 (73 %)
Qwen2.5-VL-7B	~14 GB	144/180 (80 %)
Qwen3-VL-8B	~18 GB	150/180 (83 %) – seleccionado

180 (ver Tabla 5), por lo que se descartaron en esta etapa temprana sin continuar su calibración.

Antes de comparar el desempeño de los candidatos restantes, era necesario asegurar que cada uno evaluara de la mejor forma posible las respuestas de los modelos VLM. Para eso se decidió someter el *prompt* de cada candidato a un proceso de mejora continua, orientado a maximizar su *exact match*: es decir, a sacarle el mejor provecho posible a las capacidades de los modelos que ya mostraban buenos resultados, antes de comparar entre ellos cuál sería el juez definitivo.

Este proceso se automatizó de la siguiente forma:

- Se ejecutaba una prueba con la versión vigente del *prompt* sobre todos los modelos candidatos a juez.
- Se revisaban los resultados para identificar los mejores modelos y las preguntas concretas en las que el juez fallaba al evaluarlos.
- En base a esas fallas, se agregaban reglas específicas al *prompt* para corregirlas. Por ejemplo, si se detectaba que el juez fallaba al validar la extracción de datos exactos, se añadía una regla puntual para evitar ese tipo de error desde la siguiente versión del *prompt*.
- Este ciclo se repetía de forma iterativa, generando una nueva versión del *prompt* en cada pasada.

Este proceso se automatizó mediante una IA que leía los resultados de cada iteración e inyectaba directamente las mejoras correspondientes al *prompt* siguiente.

Este proceso de mejora continua se aplicó en paralelo sobre los tres candidatos —gemma3-27b (variante cuantizada de 27 mil millones de parámetros), gemma-3-12b y MPrometheus-14B— a lo largo de 23 versiones de *prompt* (v1 a v23; el detalle completo de cada versión se presenta en el Anexo 5.2).

La versión 9 fue la mejor para gemma3-27b, alcanzando un 72,5 % de *exact match* gracias a una regla que distingue si el lenguaje condicional de una respuesta (“probablemente...”) afecta la información principal solicitada o solo un detalle secundario.

Aplicando a cada candidato su mejor versión de *prompt* alcanzada mediante este proceso, se comparó su desempeño contra el mismo set de 40 respuestas previamente evaluadas por un humano (10 por cada nivel de la rúbrica), siguiendo el esquema de *LLM-as-a-judge-evaluation* descrito en el Capítulo 2: el porcentaje de *exact match* (que el LLM asigne exactamente el mismo puntaje que el humano) y el error absoluto medio (MAE) de esos puntajes respecto de los del humano. Gracias a ese proceso de mejora continua, gemma3-27b obtuvo el mejor resultado del grupo y fue elegido como juez definitivo (ver Tabla 5).

Tabla 5: Comparación de los modelos LLM probados como candidatos a juez, incluyendo el descarte preliminar y los tres candidatos evaluados formalmente.

Fuente: Elaboración propia.

Modelo	Etapas	Puntaje	MAE
Qwen3-32B	Descarte preliminar	6,11 % (11/180)	–
Qwen3-14B (razonamiento)	Descarte preliminar	27,78 % (50/180)	–
Llama 3.1 8B	Descarte preliminar	36,11 % (65/180)	–
Phi-4-mini	Descarte preliminar	41,11 % (74/180)	–
Qwen3-14B	Descarte preliminar	43,89 % (79/180)	–
Ministral 8B	Descarte preliminar	55,56 % (100/180)	–
MPrometheus-14B	Candidato final	57,5 % (23/40)	–
gemma-3-12b	Candidato final	62,5 % (25/40)	–
gemma3-27b	Candidato final	72,5 % (29/40) - seleccionado	0,450

Una revisión posterior, ya con gemma3-27b elegido como juez, detectó que parte de ese resultado se debía a un sesgo de contexto: los ejemplos didácticos del *prompt* v9 usaban dominios que calcaban preguntas reales del conjunto de prueba (carrito de compras, acceso a una cuenta, ofertas del día), lo que podía inflar el acuerdo con el humano por memorización de dominio en lugar de por reglas generalizables.

Se dedicó entonces una nueva serie de versiones (v14 a v23) a reescribir todos los ejemplos hacia dominios neutros, sin ningún solapamiento con las preguntas reales; el resultado cayó primero y se fue recuperando hasta empatar el récord histórico en la versión 20 (72,5 %), esta vez mediante reglas generalizables. Al comparar el MAE de ambas versiones empatadas, la v9 resultó levemente mejor (0,450 frente a 0,475 de la v20). A pesar de esto, se eligió la versión 20 como la definitiva, ya que su desempeño es más robusto y generalizable, y no depende de un sesgo de contexto que podría no repetirse en otros conjuntos de prueba.

El objetivo inicial era alcanzar un 75 % de *exact match*; no se logró superar el 72,5 %, techo empírico confirmado tras las 23 versiones probadas.

3.6.5. Resultado de la selección

Elegido gemma3-27b como juez, se corrió nuevamente el *benchmark* con él sobre el universo completo de modelos candidatos. Dado que no se logró un *exact match* de 80 % o superior, se concluye que el instrumento de *benchmark* automatizado no es lo bastante confiable para elegir de forma autónoma un único mejor modelo. Sin embargo, sí resulta útil para acotar el universo de modelos posibles. A modo ilustrativo, el filtro por la etiqueta *image-text-to-text* en Hugging Face —la clasificación estándar para modelos VLM— arroja más de 30.000 modelos disponibles públicamente [Hugging Face, 2026]; el *benchmark* automatizado permitiría reducir ese universo al orden de los 10 mejores modelos candidatos, antes de pasar a un análisis manual sobre ese conjunto acotado. Ese filtrado reduce drásticamente el trabajo de revisión manual, que de otra forma sería inviable sobre el universo completo; lo que se realiza de forma manual sobre ese conjunto acotado es, en esencia, lo mismo que hace el juez LLM de forma automatizada, con la diferencia de que en ese instrumento —el criterio humano— sí se puede confiar. Sobre el conjunto de 10 modelos preseleccionados de esta forma se eligió finalmente Qwen3-VL-8B.

CAPÍTULO 4

VALIDACIÓN DE LA SOLUCIÓN

4.1. ENFOQUE DE LA VALIDACIÓN

La validación de este trabajo combina dos ejes complementarios. El primero consiste en un conjunto de métricas técnicas, calculadas directamente por el autor sobre el sistema construido: la exactitud del reconocimiento de voz (STT), la latencia del *backend*, la latencia de captura de pantalla y la pertinencia de las respuestas entregadas por el modelo. El segundo consiste en pruebas con usuarios reales con discapacidad visual, relatadas como entrevistas: se presenta el sistema al usuario, este intenta un conjunto de tareas reales de interpretación de imágenes en sitios web integradas a su flujo de trabajo habitual con su lector de pantalla, y valora cada interacción.

Dado que el número de usuarios disponibles para las pruebas es reducido (3 personas) y su selección no siguió una metodología de muestreo estadístico, las entrevistas no se presentan como un procedimiento estadístico ni sus resultados se usan para declarar la solución “validada” o “no validada” en términos porcentuales. Su objetivo es más acotado: mostrar, mediante casos concretos, si el sistema funciona o no en la práctica y de qué forma, complementando así el eje de métricas técnicas con la perspectiva de uso real.

4.2. MÉTRICAS TÉCNICAS

Estas métricas se calculan directamente por el autor sobre el funcionamiento del sistema, sin depender de la evaluación de terceros ni de la opinión de los usuarios.

4.2.1. Exactitud del reconocimiento de voz (STT)

Se mide sobre un conjunto de N palabras habladas por el usuario durante las pruebas en que la pregunta se formuló por voz (ver Capítulo 3), contabilizando cuántas de ellas fueron transcritas incorrectamente por el motor de STT utilizado por la extensión.

pendiente: definir N , ejecutar la medición y reportar la tasa de error resultante (palabras erróneas / N).

4.2.2. Latencia del *backend*

Se mide el tiempo transcurrido entre que la extensión envía una consulta (captura de pantalla y pregunta) al *backend*, y el momento en que este devuelve la respuesta en texto.

pendiente: ejecutar la medición sobre un conjunto de consultas y reportar el tiempo promedio y su dispersión.

4.2.3. Latencia de captura

Se compara el tiempo que toma la extensión en capturar el *viewport* visible de la pantalla frente al tiempo que toma capturar la página completa mediante el mecanismo de *scroll & stitch* (ver Capítulo 3).

pendiente: ejecutar la medición y reportar ambos tiempos.

4.2.4. Pertinencia de las respuestas

Se evalúa qué tan pertinente resulta la respuesta entregada por el sistema respecto de lo que el usuario efectivamente preguntó, reutilizando el criterio de la rúbrica cualitativa definida en el Capítulo 3 (ver 3.6), aplicado esta vez sobre las consultas reales realizadas durante las pruebas con usuarios.

pendiente: ejecutar la medición sobre las consultas de las sesiones y reportar los resultados.

4.3. PRUEBAS CON USUARIOS: ENTREVISTAS

4.3.1. Diseño

Se estableció un número objetivo de entre 3 y 5 usuarios. Se exigió que todos presentaran discapacidad visual severa o ceguera total, que fueran usuarios habituales de lectores de pantalla y que tuvieran un nivel de manejo computacional intermedio o avanzado.

Como entorno de pruebas se utilizó un subconjunto de los sitios evaluados en el *benchmark* (ver 3.6): al menos 2 sitios de categorías distintas y al menos 2 tareas por cada sitio. Además, se exigió formular la pregunta por voz al menos una vez.

El detalle de las tareas por sesión se presenta en una tabla en el anexo. REFERENCIA PENDIENTE: insertar ?? a la tabla del anexo correspondiente.

4.3.2. Instrumento: puntaje de satisfacción

Al terminar cada tarea se le pregunta al usuario su nivel de satisfacción con la respuesta obtenida, en un número entero entre 1 y 5, denominado puntaje de satisfacción (PS): 1 corresponde a insatisfacción total (el relator de pantalla alucinó información o su respuesta fue de nula ayuda para completar la tarea), 3 a satisfacción parcial (el relator de pantalla cumplió con lo mínimo solicitado, pero hay espacio de mejora) y 5 a satisfacción total (la respuesta fue la esperada y ayudó a completar la tarea sin problemas). Este puntaje se reporta como parte del relato de cada entrevista, sin agregarlo en un promedio o porcentaje global.

4.3.3. Participantes

La validación se realizó finalmente con 3 participantes, dentro del rango objetivo definido. Los tres presentan ceguera total y usan JAWS como lector de pantalla. Se diferencian entre sí en las condiciones asociadas a su discapacidad visual y en su nivel de manejo computacional (uno con nivel medio y dos con nivel avanzado).

4.3.4. Sesiones

Sobre estos 3 participantes se realizaron 4 sesiones en total, ya que P2 realizó dos sesiones. De ellas, en 3 la pregunta se formuló por escrito y en 1 por voz.

4.3.5. Caso 1: primer usuario (P1)

La primera sesión se realizó con P1 (nivel de manejo computacional medio), con la pregunta formulada por escrito.

PENDIENTE: narrar en formato de entrevista la sesión de P1 – tareas concretas intentadas, cómo respondió el relator de pantalla en cada una, y el puntaje de satisfacción (1-5) que P1 asignó a cada tarea. Al ser la primera sesión ejecutada, sirve como caso ilustrativo de referencia sobre la impresión inicial del sistema frente a un usuario real.

4.3.6. Casos 2 a 4: P2 (dos sesiones) y P3

PENDIENTE: narrar en el mismo formato de entrevista (tareas intentadas, respuesta del sistema, puntaje de satisfacción por tarea) las sesiones de P2 – su primera sesión con la pregunta formulada por escrito y su segunda, la única de las 4 con la pregunta formulada por voz – y de P3.

A modo de resumen de las 19 tareas intentadas a lo largo de las 4 sesiones, las puntuaciones de satisfacción se concentraron mayoritariamente en el rango medio-alto (3 a 5), aunque también hubo tareas con puntajes bajos, coherentes con los modos de falla identificados a continuación.

4.4. ANÁLISIS DE PATRONES

A partir de la revisión de los registros de las sesiones se identificaron patrones en el desempeño del sistema. Por categoría de sitio, el mejor desempeño se observó en sitios de *e-commerce* y redes sociales, mientras que el peor desempeño se dio en sitios educativos, particularmente en portales de gestión interna que requieren autenticación. Por tipo de tarea, el sistema tuvo buen desempeño en tareas descriptivas o de extracción de información –por ejemplo, “qué es”, “qué hay” o “describe”–, y un desempeño deficiente en tareas procedimentales o de navegación –por ejemplo, “cómo hago”, “dónde voy” o solicitudes de pasos a seguir–.

Entre los modos de falla identificados se cuentan: un sesgo de perspectiva vidente en las respuestas del modelo, alucinaciones cuando el elemento consultado no está visible en la captura, descripciones superficiales, y fallas del mecanismo de *scroll & stitch* (ver Capítulo 3, sección de extensión de navegador) al capturar ventanas emergentes (no *pop-ups*, sino ventanas independientes usualmente para una sola función, como por ejemplo pagos *online* o visualizar documentos).

En la modalidad de voz se observó un comportamiento particular: se reportó una satisfacción más alta en la misma tarea ejecutada por texto incluso cuando esta no se completaba. Esto sugiere que la experiencia de interacción por voz tiene un valor intrínseco, independiente del éxito de la tarea.

4.5. RETROALIMENTACIÓN DE LOS USUARIOS

Los participantes aportaron las siguientes sugerencias de mejora durante las sesiones:

- Contar con una versión móvil y compatible con aplicaciones, no solo con navegadores de escritorio.
- Que el chat mantuviera memoria de la conversación, para poder seguir preguntando sobre un mismo elemento sin repetir contexto.
- Poder acceder o navegar directamente a un enlace ya identificado por el sistema.
- Mejorar el *feedback* del sistema de entrada por voz, para que el usuario sepa si lo que dice se está capturando correctamente.

4.6. CONCLUSIÓN DE LA VALIDACIÓN

Esta validación no busca declarar la solución “aprobada” en base a un umbral estadístico, sino reunir evidencia, desde dos ángulos complementarios, de si el relator de pantalla funciona en la práctica. **PENDIENTE: una vez completadas las métricas técnicas (exactitud de STT, latencia del *backend*, latencia de captura y pertinencia de las respuestas), sintetizar aquí sus resultados.** Por el lado de las entrevistas, los casos relatados muestran un sistema que responde razonablemente bien en tareas descriptivas y de extracción de información, particularmente en sitios de *e-commerce* y redes sociales, y que tiene un desempeño más débil en tareas procedimentales y en sitios que requieren autenticación.

Los modos de falla identificados en la sección de análisis de patrones y las sugerencias reportadas por los propios participantes se abordan como trabajo futuro.

CAPÍTULO 5

CONCLUSIONES

Las Conclusiones son, según algunos especialistas, el aspecto principal de una memoria, ya que reflejan el aprendizaje final del autor del documento. En ellas se tiende a considerar los alcances y limitaciones de la propuesta de solución, establecer de forma simple y directa los resultados, discutir respecto a la validez de los objetivos formulados, identificar las principales contribuciones y aplicaciones del trabajo realizado, así como su impacto o aporte a la organización o a los actores involucrados. Otro aspecto que tiende a incluirse son recomendaciones para quienes se sientan motivados por el tema y deseen profundizarlo, o lineamientos de una futura ampliación del trabajo.

Todo esto debe sintetizarse en al menos 5 páginas.

ANEXOS

En los Anexos se incluye todo aquel material complementario que no es parte del contenido de los capítulos de la memoria, pero que permiten a un lector contar con un contenido adjunto relacionado con el tema.

5.1. ENCUESTA SOBRE TECNOLOGÍAS PARA PERSONAS CIEGAS

Este anexo detalla el instrumento mencionado en el Capítulo 1 (Definición del problema), aplicado para validar empíricamente la problemática social antes de proponer una solución.

Tabla 6: Ficha técnica de la encuesta.
Fuente: Elaboración propia.

Instrumento	“Encuesta sobre tecnologías para personas ciegas”
Plataforma	Google Forms
Validación previa	Director de FUNDALURP (Fundación de Lucha contra la Retinitis Pigmentosa)
Canal de distribución	Correo electrónico, a personas asistidas por FUNDALURP o vinculadas a ella
Período de aplicación	26 de febrero de 2026 – 11 de marzo de 2026
Respuestas obtenidas	9 (N=9)
Encuestados que usan lector de pantalla	7 de 9 (78 %)

La encuesta se estructuró en cuatro bloques temáticos:

- **Datos generales y de contacto:** marca temporal de la respuesta, correo electrónico opcional (para diferenciar participantes y evitar respuestas duplicadas), e interés en ser contactado a futuro en el contexto del proyecto.
- **Perfil del encuestado:** género; año de nacimiento (usado para estimar el contexto tecnológico según la época); condición que causa la discapacidad visual (la más severa o antigua, si el encuestado padece más de una); forma en que la condición afecta la visión; nivel de discapacidad visual; grado de manejo computacional actual. Si la discapacidad fue adquirida en una etapa avanzada de la vida (y no de nacimiento), se agregan dos preguntas adicionales: el grado de manejo computacional y el tipo de tecnologías que dominaba antes de la pérdida de visión.
- **Uso de lectores de pantalla:** si el encuestado conoce qué es y qué hace un lector de pantalla; si emplea uno para utilizar su computador; cuáles conoce (aunque no los

use); cuál usa como lector principal; y qué impacto tuvo en su vida comenzar a usarlo. Las preguntas de este bloque y del siguiente solo se muestran a quienes reportan emplear un lector de pantalla.

- **Dificultades y funcionalidades:** dificultades típicas al utilizar un lector de pantalla en general; dificultades típicas al navegar sitios web específicamente con uno; selección, de una lista cerrada, de las 3 funcionalidades que le parecerían más útiles; una pregunta abierta opcional para sugerir una funcionalidad adicional no considerada en la lista; y una pregunta abierta opcional de cierre para comentarios sobre la encuesta.

5.1.1. Respuestas completas

A continuación se listan las 9 respuestas completas de la encuesta. Cada participante se identifica con un código (P1 a P9) en lugar de su correo electrónico, dato personal que se omite por completo. El orden de los participantes corresponde al orden cronológico de respuesta.

Dos preguntas del formulario usan categorías cerradas cuyo texto completo se abrevia en la tabla para no repetirlo en cada fila; su definición completa es la siguiente:

- **Nivel de discapacidad visual:** *Moderada* (dificultad para leer textos estándar y para reconocer rostros a corta distancia, incluso con corrección); *Severa* (no logra leer textos impresos convencionales y requiere apoyos específicos para la orientación y el desplazamiento en espacios públicos); *Ceguera total* (no presenta visión funcional y no puede leer textos sin el uso de apoyos como lectores de pantalla u otros sistemas de accesibilidad).
- **Manejo computacional:** *Básico* (utiliza el computador para navegar por la web, enviar correos y acceder a redes sociales); *Intermedio* (utiliza el computador de forma cotidiana y maneja herramientas de productividad como Word y Excel); *Avanzado* (utiliza software especializado para el trabajo y realiza ajustes en configuraciones del sistema o de las aplicaciones); *Experto* (domina el uso del computador y es capaz de enseñar a otras personas cómo utilizarlo).

Tabla 7: Respuestas completas de la encuesta, anonimizadas. Fuente: Elaboración propia.

Campo	Respuesta
Participante P1	
Género	Femenino
Año de nacimiento	1974
Condición	Retinitis pigmentaria
Cómo afecta la visión	Ceguera total

Campo	Respuesta
Nivel de discapacidad visual	Ceguera total
Manejo computacional actual	Avanzado
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Experto
Tecnologías dominadas antes de la pérdida	Navegar por sitios web, redes sociales (Facebook, Twitter, Instagram), correo electrónico (Gmail, Outlook), tiendas en línea (Falabella, AliExpress), banca en línea, videoconferencias (Zoom, Google Meet), productividad (Word, Excel), manejo de carpetas y archivos del sistema, programas de diseño gráfico, edición audiovisual o arquitectura (Adobe, AutoDesk)
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	JAWS, NVDA, TalkBack, Narrador
Lector de pantalla principal	NVDA
Impacto de comenzar a usarlo	(sin respuesta)
Dificultades al utilizar un lector de pantalla	Incapacidad de interpretar elementos estrictamente visuales como imágenes; incapacidad de leer notificaciones o <i>pop-ups</i> independientes de la estructura del sitio o ubicados en otra sección de la página que no está seleccionada en ese momento
Dificultades al navegar sitios web	Recuadros de texto a rellenar sin etiqueta o que no informan qué información se debe ingresar; obligación de usar el ratón para interactuar con el sitio; pruebas de verificación humana estrictamente visuales
Funcionalidades más útiles seleccionadas	Describir e interpretar imágenes, fotografías y gráficos; solicitar por voz la navegación directa a una sección del sitio (ej. "Llévame al precio"); lectura de notificaciones o etiquetas dinámicas
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	(sin respuesta)
Participante P2	
Género	Femenino
Año de nacimiento	1975

Campo	Respuesta
Condición	Retinitis pigmentaria
Cómo afecta la visión	Visión de túnel o periférica reducida; pérdida de visión central; sensibilidad a la luz
Nivel de discapacidad visual	Severa
Manejo computacional actual	Intermedio
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Intermedio
Tecnologías dominadas antes de la pérdida	Navegar por sitios web, redes sociales (Facebook, Twitter, Instagram), correo electrónico (Gmail, Outlook), tiendas en línea (Falabella, AliExpress), banca en línea, videoconferencias (Zoom, Google Meet), productividad (Word, Excel), manejo de carpetas y archivos del sistema
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	No
Participante P3	
Género	Masculino
Año de nacimiento	1965
Condición	Degeneración Macular Asociada a la Edad (DMAE)
Cómo afecta la visión	Visión borrosa o nublada en general; zonas oscuras o manchas flotantes; sensibilidad a la luz
Nivel de discapacidad visual	Severa
Manejo computacional actual	Básico
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Intermedio
Tecnologías dominadas antes de la pérdida	Navegar por sitios web, redes sociales (Facebook, Twitter, Instagram), correo electrónico (Gmail, Outlook), tiendas en línea (Falabella, AliExpress), banca en línea, manejo de carpetas y archivos del sistema, programas de diseño gráfico, edición audiovisual o arquitectura (Adobe, AutoDesk)
¿Conoce qué es un lector de pantalla?	Sí

Campo	Respuesta
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	TalkBack, Narrador
Lector de pantalla principal	VoiceOver
Impacto de comenzar a usarlo	"Fue como volver a ser más autónomo"
Dificultades al utilizar un lector de pantalla	Incapacidad de interpretar elementos estrictamente visuales como imágenes; orden de lectura ilógico que obliga a pasar por información irrelevante antes de llegar a la clave; incapacidad de leer notificaciones o <i>pop-ups</i>
Dificultades al navegar sitios web	Recuadros de texto sin etiqueta; el sistema entrega información valiosa mediante imágenes sin texto alternativo; pruebas de verificación humana estrictamente visuales
Funcionalidades más útiles seleccionadas	Obtener un resumen del contenido principal del sitio; poder hacer preguntas sobre el sitio (ej. "¿Qué puedo hacer en esta página?"); solicitar por voz la navegación directa a una sección del sitio
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	(sin respuesta)
Participante P4	
Género	Masculino
Año de nacimiento	1972
Condición	Nanofalmo
Cómo afecta la visión	Visión borrosa o nublada en general; baja visión, imposibilidad de identificar figuras, letras o detalles
Nivel de discapacidad visual	Severa
Manejo computacional actual	Básico
Origen de la discapacidad	De nacimiento
¿Conoce qué es un lector de pantalla?	No
¿Emplea un lector de pantalla?	No
Participante P5	
Género	Masculino
Año de nacimiento	1973
Condición	Opacidad de córnea (síndrome de Peters)

Campo	Respuesta
Cómo afecta la visión	Visión borrosa o nublada en general; distingue colores y bultos, aunque los colores muy oscuros los percibe como negro (ej. distingue semáforos de noche o con clima nublado)
Nivel de discapacidad visual	Severa
Manejo computacional actual	Básico
Origen de la discapacidad	De nacimiento
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	JAWS, NVDA, TalkBack
Lector de pantalla principal	NVDA
Impacto de comenzar a usarlo	"Me siento en igualdad de condiciones ante las personas sin discapacidad visual al usar un pc como ellos"
Dificultades al utilizar un lector de pantalla	Incapacidad de interpretar elementos estrictamente visuales como imágenes; orden de lectura ilógico; incapacidad de leer notificaciones o <i>pop-ups</i> ; nula reacción ante cambios en la presentación del sitio (secciones que aparecen y desaparecen); respuesta libre: "los gráficos y cuando no especifica los botones cualesquiera éstos sean"
Dificultades al navegar sitios web	El sistema entrega información valiosa mediante imágenes sin texto alternativo; instrucciones estrictamente visuales (ej. "seleccione la opción verde"); la página cambia su estructura sin avisar; obligación de usar el ratón; pruebas de verificación humana estrictamente visuales; respuesta libre: "los gráficos y botones sin descripción"
Funcionalidades más útiles seleccionadas	Describir e interpretar imágenes, fotografías y gráficos; solicitar por voz la navegación directa a una sección del sitio; lectura de notificaciones o etiquetas dinámicas
Funcionalidad adicional sugerida	"El significado de los botones y gráficos que muestran los símbolos e imágenes pero para el lector no están descritos en forma escrita, como para que nos diga: botón ingresar, la imagen representa tal o cual figura... cuestiones así por el estilo"
Comentario final	"Descripción de gestos, textos, gráficos e imágenes"
Participante P6	
Género	Masculino
Año de nacimiento	2008

Campo	Respuesta
Condición	Neuropatía óptica hereditaria de Leber
Cómo afecta la visión	Pérdida de visión central; escasa visión periférica y cambio en la percepción de colores
Nivel de discapacidad visual	Severa
Manejo computacional actual	Básico
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Básico
Tecnologías dominadas antes de la pérdida	Navegar por sitios web, videoconferencias (Zoom, Google Meet)
¿Conoce qué es un lector de pantalla?	No
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	NVDA, TalkBack
Lector de pantalla principal	TalkBack
Impacto de comenzar a usarlo	“Me ayudó a adaptarme a esta nueva normalidad de mejor forma”
Dificultades al utilizar un lector de pantalla	Orden de lectura ilógico que obliga a pasar por información irrelevante antes de llegar a la clave
Dificultades al navegar sitios web	La página cambia su estructura u organización sin avisar
Funcionalidades más útiles seleccionadas	Obtener un resumen del contenido principal del sitio; describir e interpretar imágenes, fotografías y gráficos; solicitar por voz la navegación directa a una sección del sitio
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	(sin respuesta)
Participante P7	
Género	Otro
Año de nacimiento	1982
Condición	Glaucoma
Cómo afecta la visión	(respuesta del encuestado poco clara en el formulario original: “Lucero visión”)
Nivel de discapacidad visual	Ceguera total

Campo	Respuesta
Manejo computacional actual	Intermedio
Origen de la discapacidad	De nacimiento
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	JAWS, NVDA, VoiceOver, TalkBack, Narrador, Orca
Lector de pantalla principal	VoiceOver
Impacto de comenzar a usarlo	“Es mi forma y manera de comunicarme con otros y ser productivo laboralmente”
Dificultades al utilizar un lector de pantalla	Incapacidad de interpretar elementos estrictamente visuales como imágenes; orden de lectura ilógico; incapacidad de leer notificaciones o <i>pop-ups</i> ; nula reacción ante cambios en la presentación del sitio
Dificultades al navegar sitios web	Recuadros de texto sin etiqueta; imágenes sin texto alternativo; navegar textos extensos sin estructura; instrucciones estrictamente visuales; la página cambia su estructura sin avisar; obligación de usar el ratón; pruebas de verificación humana estrictamente visuales
Funcionalidades más útiles seleccionadas	Obtener un resumen del contenido principal del sitio; poder hacer preguntas sobre el sitio; describir e interpretar imágenes, fotografías y gráficos
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	(sin respuesta)
Participante P8	
Género	Femenino
Año de nacimiento	1979
Condición	Glaucoma
Cómo afecta la visión	Ceguera total
Nivel de discapacidad visual	Ceguera total
Manejo computacional actual	Básico
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Intermedio

Campo	Respuesta
Tecnologías dominadas antes de la pérdida	Navegar por sitios web
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	JAWS, NVDA, VoiceOver, TalkBack
Lector de pantalla principal	NVDA
Impacto de comenzar a usarlo	(sin respuesta)
Dificultades al utilizar un lector de pantalla	Incapacidad de interpretar elementos estrictamente visuales como imágenes; orden de lectura ilógico; incapacidad de leer notificaciones o <i>pop-ups</i>
Dificultades al navegar sitios web	Recuadros de texto sin etiqueta; imágenes sin texto alternativo; navegar textos extensos sin estructura; instrucciones estrictamente visuales; pruebas de verificación humana estrictamente visuales
Funcionalidades más útiles seleccionadas	Obtener un resumen del contenido principal del sitio; describir e interpretar imágenes, fotografías y gráficos; solicitar por voz la navegación directa a una sección del sitio
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	(sin respuesta)
Participante P9	
Género	Femenino
Año de nacimiento	1967
Condición	Retinitis pigmentaria
Cómo afecta la visión	Visión de túnel o periférica reducida; zonas oscuras o manchas flotantes
Nivel de discapacidad visual	Moderada
Manejo computacional actual	Básico
Origen de la discapacidad	Adquirida (por enfermedad o accidente)
Manejo computacional antes de la pérdida	Básico
Tecnologías dominadas antes de la pérdida	Productividad (Word, Excel)

Campo	Respuesta
¿Conoce qué es un lector de pantalla?	Sí
¿Emplea un lector de pantalla?	Sí
Lectores de pantalla que conoce	NVDA, TalkBack
Lector de pantalla principal	NVDA
Impacto de comenzar a usarlo	“Me emocioné mucho porque, cuando escribo, el lector me va leyendo y así estoy segura de lo que voy escribiendo”
Dificultades al utilizar un lector de pantalla	Respuesta libre: “mi dificultad es que el lector no deja de hablar”
Dificultades al navegar sitios web	Navegar textos extensos sin estructura; instrucciones estrictamente visuales
Funcionalidades más útiles seleccionadas	Poder hacer preguntas sobre el sitio; describir e interpretar imágenes, fotografías y gráficos; solicitar por voz la navegación directa a una sección del sitio
Funcionalidad adicional sugerida	(sin respuesta)
Comentario final	“Me parece una iniciativa muy buena que se sigan buscando mejoras para el lector de pantalla”

5.2. CALIBRACIÓN DEL JUEZ LLM: HISTORIAL DE VERSIONES DEL PROMPT

Este anexo detalla el proceso mencionado en el Capítulo 3 (sección 3.6, “Selección del juez”): la calibración del *prompt* del juez LLM contra el conjunto de 40 respuestas evaluadas por un humano, a lo largo de 23 versiones (v1 a v23). Para el candidato gemma3-27b se incluye además el desglose de aciertos por nivel de la rúbrica (L0/L1/L2/L3, sobre un total de 10 casos por nivel).

Tabla 8: Resultados de *exact match* por versión de *prompt* y modelo juez. Fuente: Elaboración propia.

Versión	gemma3-27b (L0/L1/L2/L3)	gemma-3-12b	MPrometheus-14B
v1	sin métricas formales	–	–
v2	sin métricas formales	–	–
v3	sin métricas formales	–	–
v4	sin métricas formales	–	–
v5	70,0 % (7/3/9/9)	60,0 %	45,0 %

Versión	gemma3-27b (L0/L1/L2/L3)	gemma-3-12b	MPrometheus-14B
v6	70,0 % (4/6/8/10)	57,5 %	52,5 %
v7	67,5 % (4/6/7/10)	52,5 %	45,0 %
v8	62,5 % (3/5/7/10)	55,0 %	50,0 %
v9	72,5 % (5/6/8/10)	60,0 %	52,5 %
v10	72,5 % (4/6/9/10)	55,0 %	42,5 %
v11	70,0 % (5/5/8/10)	62,5 %	57,5 %
v12	70,0 % (4/5/9/10)	57,5 %	47,5 %
v13	70,0 % (5/5/8/10)	-	-
v14	67,5 % (3/6/8/10)	50,0 %	45,0 %
v15	42,5 % (5/8/2/2)	37,5 %	32,5 %
v16	70,0 % (6/4/8/10)	-	-
v17	70,0 % (6/4/8/10)	-	-
v18	70,0 % (6/4/8/10)	-	-
v19	67,5 % (6/3/8/10)	-	-
v20	72,5 % (6/5/8/10)	-	-
v21	70,0 % (6/6/6/10)	-	-
v22	67,5 % (6/4/7/10)	-	-
v23	70,0 % (6/6/6/10)	-	-

REFERENCIAS BIBLIOGRÁFICAS

- [Abou-Zahra y World Wide Web Consortium (W3C) Web Accessibility Initiative, 2024] Abou-Zahra, S. y World Wide Web Consortium (W3C) Web Accessibility Initiative (2024). Tools and techniques. <https://www.w3.org/WAI/people-use-web/tools-techniques/>. Actualizada el 25 de junio de 2024.
- [Bourne et al., 2021] Bourne, R. R. A., Steinmetz, J. D., Flaxman, S., y GBD 2019 Blindness and Vision Impairment Collaborators (2021). Trends in prevalence of blindness and distance and near vision impairment over 30 years: an analysis for the global burden of disease study. *The Lancet Global Health*, 9(2):e130–e143.
- [Burnett y Shuang, 2010] Burnett, D. C. y Shuang, Z. W. (2010). Speech synthesis markup language (ssml) version 1.1. Technical report, World Wide Web Consortium. W3C Recommendation, 7 de septiembre de 2010.
- [Campbell et al., 2024] Campbell, A., Adams, C., Montgomery, R. B., Cooper, M., y Kirkpatrick, A. (2024). Web content accessibility guidelines (wcag) 2.2. Technical report, World Wide Web Consortium. W3C Recommendation, 12 de diciembre de 2024.
- [Chromium Blog, 2010] Chromium Blog (2010). A year of extensions. <https://blog.chromium.org/2010/12/year-of-extensions.html>. 9 de diciembre de 2010.
- [Deque Systems, 2019] Deque Systems (2019). Quick reference guide: Jaws for windows keyboard commands. <https://dequeuniversity.com/assets/pdf/screenreaders/jaws-guide.pdf>. Versión 2019.03.11.
- [Duan et al., 2024] Duan, H., Fang, X., Yang, J., Zhao, X., Ma, Z., Qiao, Y., Li, M., Liang, T., Zhu, L., Chen, Z., y others (2024). VLMEvalKit: An open-source toolkit for evaluating large multi-modality models. *arXiv preprint arXiv:2407.11691*.
- [Freedom Scientific, 2024] Freedom Scientific (2024). Jaws – screen reading software. <https://www.freedomscientific.com/products/software/jaws/>.
- [Hugging Face, 2026] Hugging Face (2026). Models – image-text-to-text. https://huggingface.co/models?pipeline_tag=image-text-to-text. Listado filtrado por la etiqueta image-text-to-text (modelos de lenguaje visual); la cifra incluye variantes, ajustes finos y cuantizaciones de una misma arquitectura base.
- [IBM, 2024a] IBM (2024a). Generative ai vs. predictive ai: What's the difference? <https://www.ibm.com/think/topics/generative-ai-vs-predictive-ai-whats-the-difference>.
- [IBM, 2024b] IBM (2024b). What is data bias? <https://www.ibm.com/think/topics/data-bias>.

- [IBM, 2024c] IBM (2024c). What is model training? <https://www.ibm.com/think/topics/model-training>.
- [IBM, 2024d] IBM (2024d). What is overfitting vs. underfitting? <https://www.ibm.com/think/topics/overfitting-vs-underfitting>.
- [IBM, 2024e] IBM (2024e). What is speech recognition? <https://www.ibm.com/think/topics/speech-recognition>.
- [IBM, 2024f] IBM (2024f). What is training data? <https://www.ibm.com/think/topics/training-data>.
- [Mace y The Center for Universal Design, 1997] Mace, R. y The Center for Universal Design (1997). The 7 principles of universal design. Technical report, North Carolina State University. Mirror publicado por el Centre for Excellence in Universal Design (CEUD), National Disability Authority, Irlanda.
- [MDN Web Docs, 2024a] MDN Web Docs (2024a). Accessibility tree. https://developer.mozilla.org/en-US/docs/Glossary/Accessibility_tree.
- [MDN Web Docs, 2024b] MDN Web Docs (2024b). Aria. <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>.
- [MDN Web Docs, 2024c] MDN Web Docs (2024c). Webextensions. <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions>.
- [Nielsen, 1993] Nielsen, J. (1993). Response time limits: Article by jakob nielsen. <https://www.nngroup.com/articles/response-times-3-important-limits/>. Nielsen Norman Group.
- [Nurthen et al., 2023] Nurthen, J., Cooper, M., y Henry, S. L. (2023). Wai-aria 1.2. Technical report, World Wide Web Consortium (W3C). W3C Recommendation, 6 de junio de 2023.
- [NV Access, 2024] NV Access (2024). About nvda. <https://www.nvaccess.org/about-nvda/>.
- [NVIDIA, 2024] NVIDIA (2024). What is ai inference? <https://www.nvidia.com/en-us/glossary/ai-inference/>.
- [Tan et al., 2021] Tan, X., Qin, T., Soong, F., y Liu, T.-Y. (2021). A survey on neural speech synthesis. *arXiv preprint arXiv:2106.15561*.
- [Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. En *Advances in Neural Information Processing Systems*, volumen 30.
- [WebAIM, 2024] WebAIM (2024). Screen reader user survey #10 results. <https://webaim.org/projects/screenreadersurvey10/>.

- [WHATWG, 2026] WHATWG (2026). Dom standard. <https://dom.spec.whatwg.org/>. Living Standard.
- [World Health Organization, 2026] World Health Organization (2026). Blindness and vision impairment. <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>. Fact sheet, actualizada el 10 de febrero de 2026.
- [World Wide Web Consortium (W3C), 2021] World Wide Web Consortium (W3C) (2021). Forming the webextensions community group. <https://www.w3.org/community/webextensions/2021/06/04/forming-the-wecg/>. Anuncio de formación del grupo comunitario, 4 de junio de 2021.
- [World Wide Web Consortium (W3C), Web Accessibility Initiative, 2014] World Wide Web Consortium (W3C), Web Accessibility Initiative (2014). Resources on alternative text for images. <https://www.w3.org/WAI/alt/>.
- [Wu et al., 2023] Wu, J., Gan, W., Chen, Z., Wan, S., y Yu, P. S. (2023). Multimodal large language models: A survey. *arXiv preprint arXiv:2311.13165*.
- [Zheng et al., 2023] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., y Stoica, I. (2023). Judging LLM-as-a-judge with MT-bench and chatbot arena. En *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volumen 36.